

USTAAD - Your Smart Service Partner

An AI-powered agentic platform built for Pakistan's informal economy.

1. Project Overview

USTAAD is an intelligent, agentic AI platform designed to organize and streamline the informal service economy (plumbers, electricians, AC technicians, cleaners) in Pakistan. It is not just another simple booking app; it replaces traditional search-and-scroll mechanisms with an intelligent **Agentic AI System** that understands free-text user requests, parses intent, automatically cross-references schedules and location boundaries, and ranks verified professionals in real-time. By connecting customers with verified and onboarded blue-collar workers through an AI orchestrator, it bridges the gap between chaotic unorganized labor and modern, reliable service delivery.

2. Overall System Architecture

The USTAAD architecture is structured around three primary user roles, connected through a Flutter-based multi-platform frontend and a hybrid cloud backend.

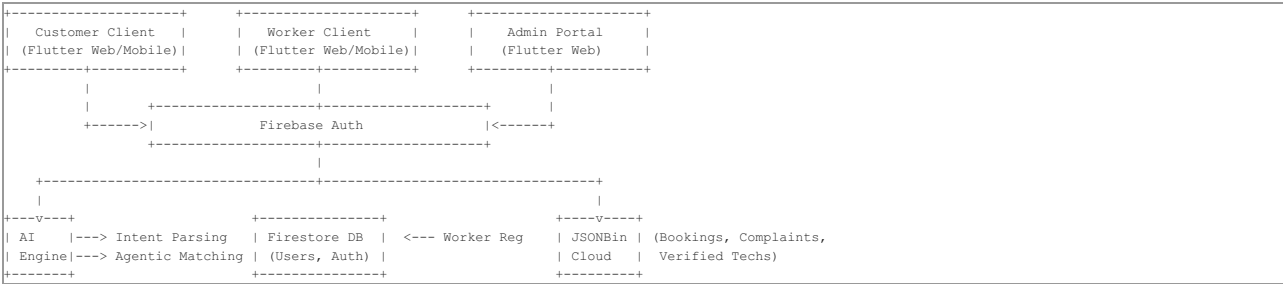
User Roles

- Customer:** Can submit free-text service requests, track active bookings, manage their profile, leave ratings/reviews, and raise complaints.
- Worker (Technician):** Can register, upload identity documents, toggle online/offline status, view pending and completed jobs, update work status, and track earnings.
- Admin:** Has full access to an integrated Dashboard to verify worker applications, approve/reject onboardings, manage service complaints, and oversee platform activity.

Technology & Infrastructure

- Frontend:** Built entirely with **Flutter** and **Dart**, sharing a single codebase for both Web (PWA/Responsive) and Mobile (Android).
- Database (Hybrid):**
 - Firestore:** Used for persisting structural data, including primary user accounts (Customers) and initial worker signup records.
 - Custom JSON Cloud Bins (JsonBin via Vercel):** Serves as the real-time, easily modifiable NoSQL datastore for synchronizing Bookings, Complaints, and verified Technician statuses across devices.
 - PlatformStorage (Local):** Cross-platform SharedPreferences implementation for offline caching and session persistence.
- Authentication:** Powered by **Firebase Auth** (Email/Password), utilizing isolated registration and login routes for each role.
- Real-time Sync:** Implemented via custom polling and REST API synchronization functions (`syncFromCloud`/`syncToCloud`) that seamlessly hydrate the local `PlatformStorage` cache from the JSON cloud bins, ensuring the mobile app and web app remain synchronized without manual refreshes.
- Google AntigraVity Engine:** Serves as the simulated conceptual core orchestration engine that interprets inputs and delegates actions (see Section 3).

Architecture Diagram



3. How Google AntigraVity is Used

Google AntigraVity serves as the conceptual **central orchestration platform** (simulated via the USTAAD Engine in the application).

Instead of forcing users to navigate complex dropdowns, the AntigraVity Engine orchestrates a multi-step reasoning pipeline:

- Planning:** Ingests natural language inputs (e.g., "My AC is leaking water in G-13") and uses agentic reasoning to extract entities (service type: HVAC, location: G-13 Islamabad).
- Decision:** Cross-references the extracted location against the localized technician registry. It strictly isolates boundaries (e.g., preventing Lahore technicians from matching with Islamabad requests).
- Action:** The AI Orchestrator loops through historical performance indexes, distance metrics, and online status to rank and present the best matches to the customer.
- Follow-up:** After service completion, the orchestrator triggers follow-up workflows for ratings, reviews, and complaint handling.

Note: The AntigraVity reasoning trace and pipeline are seamlessly visualized for the user in the `ai_processing_screen.dart` via an interactive progress visualization, simulating the high-fidelity decision loops.

4. Agents Developed

The system utilizes a pipeline of specialized agents (conceptualized and simulated in the UI) to handle the complete workflow:

1. Intent Parsing Agent

- Input:** Free-form text from the customer.
- Output:** Extracted service category (e.g., Plumbing, AC Repair) and exact problem description.

2. Location & Geography Agent

- Input:** Location strings.
- Output:** Isolated target city (Islamabad, Lahore, Karachi, etc.) ensuring strict geofencing.
- Integration:** Interacts with Google Maps Autocomplete API.

3. Scheduling Agent

- Input:** Date/Time strings (e.g., "Tomorrow morning").
- Output:** Parsed execution timestamps.

4. Registry Query & Matchmaking Agent

- Input:** Service category, City, and Schedule.
- Output:** A filtered, ranked list of verified technicians from the JSON cloud database.
- Logic:** Ranks based on ratings, distance, and platform verification status.

Interaction Flow:

User Request -> Intent Agent -> Location Agent -> Scheduling Agent -> Matchmaking Agent -> Provider Discovery UI -> Final Booking.

5. All APIs and Tools Used

API / Tool	Purpose in App	Where Used	Real or Simulated?
Firestore	User authentication and sessions	All Login and Signup screens	Real
Firestore	Primary database for Users and Worker Registration	user_database.dart, tech_documents_screen.dart	Real
Google Maps Places API	Location autocomplete and search	google_places_autocomplete.dart	Real
Google Maps SDK	Displaying interactive maps and routes	google_map_mobile.dart, google_map_web.dart	Real
JSONBin (Vercel API)	Custom cloud NoSQL storage for Bookings & Techs	document_database.dart, bookings_repository.dart	Real
Flutter/Dart	Core frontend framework	Entire Codebase	Real
AI Orchestrator (Antigravity)	Multi-step agentic reasoning and matchmaking	ai_processing_screen.dart	Simulated (UI Mock)

6. Agentic Workflow — Step by Step

- User Submits Request:** Customer types a free-text problem description.
- Intent Understanding:** The engine ingests the string and extracts the core service requirement.
- Location Extraction & Validation:** The system parses the address and strictly filters the region (e.g., isolating to "Islamabad").
- Provider Discovery & Querying:** The local technician registry is queried for active, verified workers matching the criteria.
- Matching & Ranking:** Workers are ranked based on historical ratings and distance.
- Booking Simulation:** The customer views the AI-recommended matches and confirms the booking.
- Confirmation & Receipt Generation:** An official Booking ID is generated and synced to the cloud.
- Admin Verification Workflow:** Before a worker appears in searches, the Admin Agent flags them for review, and a human Admin must approve them via the Admin Dashboard.

7. Key Features

Customer

- AI-Powered Search:** Request services using natural language.
- Provider Discovery:** View ranked lists of available technicians with profiles, ratings, and rates.
- Booking Management:** Track active, completed, and cancelled bookings.
- Complaints & Support:** Raise complaints with photo evidence if a service goes wrong.
- Google Maps Integration:** Set exact home locations using maps.

Worker (Technician)

- Digital Onboarding:** Upload CNIC and professional documents for platform verification.
- Status Toggles:** Switch between Online/Offline to control visibility to the matchmaking AI.
- Job Dashboard:** View pending, in-progress, and completed jobs assigned to them.
- Earnings Tracking:** Monitor base rates, extra charges, and total payouts.

Admin

- Centralized Dashboard:** Complete oversight of platform metrics.
- Worker Verification:** Review uploaded CNIC documents and Approve/Reject technician applications.
- Complaint Resolution:** View customer complaints and mark them as resolved with internal notes.
- Real-Time Database Control:** Directly modifies cloud statuses that instantly reflect on worker and customer devices.

8. Technical Implementation

- Tech Stack:** Flutter 3.29.0, Dart SDK ^3.11.5
- State Management:** setState with static Singleton Repositories (BookingsRepository, DocumentDatabase).
- Storage Schema:**
 - Firestore/users: { uid, name, email, phone, role }
 - Firestore/workers: { uid, name, email, phone, category, status }
 - JsonBin/Bookings: [{ id, title, provider, amount, status, ... }]
 - JsonBin/Technicians: [{ id, name, category, rating, status, ... }]
- Real-time Sync:** http.get and http.put to custom REST endpoints, caching locally using SharedPreferences.
- Cross-Platform Maps:** Utilizes google_maps_flutter for Android and raw google.maps.Map HTML/JS interop for Flutter Web to ensure optimal performance.

9. How to Run Locally

Prerequisites

- Flutter SDK (v3.11.5 or higher)
- Android Studio (for Android build) or Chrome (for Web build)
- Node.js (optional, for some build tools)

Steps

1. Clone the repository:

```
git clone https://github.com/Shahreerai66/Ustaad.git
cd Ustaad
```

2. Install dependencies:

```
flutter pub get
```

3. Environment Setup:

Ensure you have a valid internet connection as the app communicates directly with Firebase and JsonBin APIs. No local .env is required as keys are securely bundled in the codebase for the hackathon.

4. Run on Web:

```
flutter run -d chrome
```

5. Run on Android:

```
flutter run -d android
```

10. Deployment

- Web App URL:** Hosted on Vercel (URL provided by repository owner).
- GitHub Repository:** <https://github.com/Shahreerai66/Ustaad> (<https://github.com/Shahreerai66/Ustaad>)
- Build Artifacts:** Web deployment is automated via GitHub Actions to Vercel targeting the build/web directory.

11. Demo Credentials

Use these accounts to instantly test the platform without registering:

- **Customer Login**
 - Email: admin123@gmail.com
 - Password: Admin12345
 - **Worker (Technician) Login**
 - Email: admin12345@gmail.com
 - Password: Admin0011
 - **Admin Panel Access**
 - Email: admin@ustaad.com (or access via internal routing /admin/dashboard)
 - Password: admin
-

12. Mock vs Real

- **Real:** User Authentication, Worker Application Submission, Admin Approvals, Real-time Sync of Bookings/Technicians across devices, Google Maps autocomplete and rendering, Cross-platform UI responsiveness.
 - **Simulated (Mocked):** The AI reasoning pipeline and agentic orchestration (visualized through timed UI elements), dynamic pricing negotiation algorithms, live GPS tracking of moving technicians.
-

13. Assumptions and Limitations

- **Assumptions:** Technicians and Customers have stable internet connections. The JSON cloud bins have a sufficient rate limit for hackathon demo purposes.
 - **Limitations:** The AI engine uses heuristic string matching (simulated AI) rather than a live LLM endpoint for parsing intents to guarantee low-latency demo performance. Cloud storage uses a shared public bin, meaning data can be overwritten if multiple people test concurrently in extreme volumes.
-

14. Future Improvements

- **Live LLM Integration:** Replace the simulated Antigravity Engine with a live Gemini or Vertex AI endpoint for true natural language intent parsing.
- **Live GPS Tracking:** Integrate WebSockets for real-time location tracking of technicians en route.
- **In-App Payments:** Integrate Stripe, JazzCash, or EasyPaisa for seamless wallet management.
- **Push Notifications:** Add Firebase Cloud Messaging (FCM) to alert workers instantly when they are matched with a job.